

Hermes Agent on a Personal Computer: Security, Sandboxing, Automation and Reliability Lessons from a Windows/Docker Practitioner Case Study

Frankie Mak

MBA | Graduate Certificate in Cyber Security

Independent Researcher and Technology Practitioner

Australia

Contact: frankiemak.research@gmail.com

Version 1.0 - Public Release | 24 September 2026

Independent practitioner case study; not presented as peer-reviewed experimental research.

Abstract

This report documents a practitioner case study of deploying and hardening the open-source Hermes Agent on a personal Windows computer. The investigation began with a host-security question - whether an autonomous AI agent executing terminal and file operations locally could safely coexist with ordinary personal computing - and evolved into a multi-layer debugging exercise involving Docker sandboxing, Windows path translation, scheduled automation, NAS access control, gateway lifecycle, Telegram integration, email delivery, and Unicode/UTF-8 corruption. The case demonstrates that agent security is not reducible to prompt restrictions or approval dialogs. Hermes documentation explicitly identifies OS-level isolation as the load-bearing security boundary and distinguishes local execution, terminal-backend isolation, and whole-process containment. The case also exposes an operational class of Windows-specific failures in which path semantics, subprocess encoding, gateway readers, and cron execution become part of the security and reliability surface. The report proposes a least-privilege architecture in which external messaging is authorized by allowlists, model/tool execution is containerized, host data is mounted only when required and preferably read-only, credentials are scoped rather than forwarded wholesale, and scheduled tasks are treated as production automation with deterministic verification and recovery procedures.

Keywords: agentic AI; Hermes Agent; Docker sandbox; Windows; OS-level isolation; prompt injection; excessive agency; cron automation; Telegram bot security; UTF-8; NAS; least privilege; AI system security

Contents

Abstract

1. Research Scope and Evidence Basis
2. Research Questions
3. Conceptual Security Model
4. Chronology of the Case
5. Native Windows Deployment: Security Implications
6. Docker Sandboxing: What It Improves and What It Does Not
7. Scheduled Automation as an Agentic Risk Multiplier
8. Windows Path Translation as a Reliability and Security Problem
9. Windows Encoding and Mojibake
10. Gateway and Telegram: From Local Tool to Remote Service
11. Security Threat Analysis
12. Observed Architecture and Secure Target Architecture
13. Engineering Lessons
14. Recommended Future-State Control Set
15. Discussion: What This Case Contributes
16. Conclusion

References

Appendix A - Public-Release Privacy Checklist

Appendix B - Proposed Reproducibility and Validation Test Matrix

1. Research Scope and Evidence Basis

The case study covers a multi-day configuration and debugging sequence undertaken in September 2026. The researcher supplied 19 private ChatGPT conversation URLs covering the installation, security assessment, Docker migration, cron jobs, gateway, Telegram, NAS, encoding and post-run verification. Those conversation pages are login-gated in the current web environment and therefore cannot be independently retrieved as public source documents. Accordingly, this report uses the retained technical conversation context as the primary incident record and cross-checks architectural claims against current Hermes documentation, the public Hermes repository, public issue reports, OWASP GenAI guidance, and Docker documentation. The case narrative should therefore be read as an evidence-bounded practitioner study rather than a forensic reconstruction of every original message.

The study is descriptive and methodological rather than statistically generalizable. The observed failures, remediation steps and validation outcomes describe one practitioner deployment and are used to derive architectural lessons and a reproducible test framework. Where current vendor or project documentation is cited, those statements describe the documented behavior of the cited version or source at the time of access.

As of 24 September 2026, the Hermes repository lists v0.21.3 (v2026.9.14) as the latest tagged release visible in the release history reviewed for this report. The release history shows rapid ongoing development across gateway, session storage, cron and related components, reinforcing the importance of explicit version pinning in reproducible security research. [1]

2. Research Questions

1. What security boundary exists when Hermes Agent is executed natively on a personal Windows host?
2. What security properties are gained - and what new failure modes are introduced - by moving command execution into Docker?
3. How do scheduled agentic tasks amplify configuration defects because they execute autonomously and repeatedly?
4. Which classes of observed Windows path, encoding, gateway and messaging failures should be treated as reliability issues, security issues, or both?
5. What reference architecture best reduces the blast radius of prompt injection, excessive agency, credential exposure and filesystem mistakes in a personal deployment?

3. Conceptual Security Model

Hermes currently describes a defense-in-depth model spanning user authorization, dangerous-command approval, file-write safety, container isolation, credential filtering, context-file scanning, cross-session isolation and input sanitization. Its SECURITY.md goes further: the project treats OS-level isolation as the only load-bearing security boundary against an adversarial LLM. In-process approval gates and string scanners are described as heuristics rather than containment. [2], [3]

This distinction is fundamental to personal deployments. A local backend may run with the same filesystem and process privileges available to the user account. By contrast, a Docker terminal backend can confine shell and file-tool execution to a container, although the agent process, MCP subprocesses, skills, plugins and hooks may remain outside that boundary when Hermes itself is still running on the host. Whole-process wrapping is therefore materially stronger when the agent consumes untrusted web, email, multi-user messaging or third-party tool content. [2]

4. Chronology of the Case

Table 1. Chronology of the practitioner case.

Phase	Area	Observed evidence	Engineering interpretation
Phase 1	Native host installation	The initial security concern was broad host reach: local terminal/file/code execution could operate under the Windows user context.	Reframed the problem as an OS boundary problem, not a prompt-writing problem.
Phase 2	Docker terminal sandbox	Docker Desktop/WSL2 state, container lifecycle, working-directory mapping and host/container path differences became operational dependencies.	Adopted a persistent Docker sandbox for command/file execution and began explicit mount verification.
Phase 3	Cron automation	Four recurring jobs became the main production-like workload: ASX close, US close, catch-up verification, and supermarket specials.	Each job was treated as an independently testable automation unit.
Phase 4	NAS read-only objective	Windows mapped-drive access and container access diverged; bind mounting the mapped drive did not behave as expected, while a CIFS-volume attempt encountered permissions/session conflicts.	Separated Windows access, Docker access, and authorization identity into distinct layers.
Phase 5	Gateway anomaly	Gateway warnings and platform configuration issues appeared during the Telegram integration stage. Temporarily disabling the problematic platform narrowed the fault domain.	Used fault isolation rather than changing many components simultaneously.
Phase 6	Encoding failures	Mojibake and Unicode problems affected supermarket categories, subprocesses, cron output and gateway-related behavior.	Promoted UTF-8 from a cosmetic concern to a system-wide reliability requirement.
Phase 7	Messaging and email	Telegram became both a control surface and a delivery surface; email remained the reporting channel for market and supermarket workflows.	Separated authorization, execution, and delivery verification.
Phase 8	Controlled target architecture	The deployment converged on explicit boundaries between gateway, agent core, sandbox, cron, data mounts and external delivery.	Future work focuses on reproducibility, monitoring, minimal privileges and version control.

5. Native Windows Deployment: Security Implications

Hermes documentation states that the native Windows install runs without WSL or Docker and that the default local backend provides no OS-level isolation. The configuration documentation also warns that the agent has the same filesystem access as the user account. [4], [5] This means a personal installation should be threat-modelled as a privileged local automation process rather than as a conventional chatbot.

The important security property is not whether the model is locally hosted or remotely hosted. Even when inference is remote, the agent runtime may still have local authority to execute commands, read files, write files, invoke tools,

and communicate externally. The trust envelope is therefore defined by the runtime and its credentials, not by where the language model itself is physically executed. [2]

6. Docker Sandboxing: What It Improves and What It Does Not

Hermes documents Docker hardening features including dropped Linux capabilities, no-new-privileges, PID limits and isolated temporary filesystems. It also states that the current working directory is not automatically mounted into the Docker sandbox; host paths require explicit configuration. [3], [5] These measures reduce the blast radius of LLM-generated shell and file operations.

However, a mount is a deliberate reintroduction of reach across the trust boundary. A writable bind mount can turn a containerized agent into a host data mutator. Forwarded environment variables can turn the sandbox into a credential disclosure channel. A Docker socket or equivalent host-control interface would create an even larger trust collapse. The correct mental model is therefore not "Docker makes Hermes safe" but "Docker provides a boundary whose strength depends on the mounts, identities, capabilities and network paths configured around it."

Docker itself documents read-only bind mounts as a supported mode, and rootless mode reduces the privileges of the daemon and containers. [10], [11] For personal agent deployments, these are useful defense-in-depth controls when compatible with the intended workload.

7. Scheduled Automation as an Agentic Risk Multiplier

Four recurring jobs were central to the case. Their identifiers are deliberately generalized in this public report as JOB-ASX, JOB-US, JOB-CATCHUP and JOB-SUPERMARKET. The original identifiers are omitted because they are deployment-specific and not required for reproducibility.

Table 2. Generalized scheduled workloads in the case.

Job	Schedule	Purpose	Observed failure classes
JOB-ASX	Weekdays 16:00 local-market schedule	Market close data, logs, CSV, email delivery	Working-directory correctness; market-day logic; delivery validation.
JOB-US	20:00 UTC	US market close data, logs, CSV, email delivery	Execution environment and path verification.
JOB-CATCHUP	Every 2 hours	Detect and recover missing market reports	Idempotency, duplicate-run avoidance, delivery safety.
JOB-SUPERMARKET	Weekly Saturday 08:00	Retail specials extraction, categorization, output, email	Missed delivery, output-directory anomalies, encoding/mojibake.

Hermes cron jobs run in fresh agent sessions and therefore do not inherit conversational memory. The official cron documentation states that the scheduler loads jobs, starts a fresh agent session, runs the prompt, delivers the final response, and updates run metadata. This has two security implications: the prompt must contain all necessary operational context, and the job itself must be narrowly scoped because it will execute without an interactive human at every step. [6]

8. Windows Path Translation as a Reliability and Security Problem

The case repeatedly encountered the difference between a path that exists on Windows and a path that is resolvable inside a Docker/cron execution context. A host path can become a container path, a shell argument, a JSON string, or a script parameter. Each layer may interpret backslashes differently. A syntactically valid path at one boundary is not evidence that it is semantically valid at the next.

```
[Host filesystem]
<WORKSPACE>\Shares\fetch_prices.py

[Container view]
/workspace/.../fetch_prices.py

[Risk boundary]
Windows path -> JSON/string -> shell/parser -> container filesystem
```

This case aligns with a class of public Hermes Windows issues. Issue #60857 reports Windows cron script paths losing backslashes, converting <drive>:\Users-like paths into malformed forms during execution. The match is conceptual rather than forensic; the present case does not have the original stack trace needed to prove the exact same code path. [12]

9. Windows Encoding and Mojibake

Encoding defects were not merely presentation bugs. They affected extracted product categories, subprocess output, scheduled task delivery and gateway behavior. A modern agent system crosses multiple encoding boundaries: web content, Python strings, filesystem bytes, child processes, gateway pipes, Telegram API payloads and email MIME bodies.

Hermes currently documents a Windows UTF-8 initialization path that switches the console to UTF-8, configures Python stdio for UTF-8, and propagates PYTHONIOENCODING/PYTHONUTF8 to child processes. Public Hermes issues have separately documented Unicode failures in subprocesses, gateway readers and script-only cron execution on Windows. [7], [13], [14], [15] The case therefore supports an explicit engineering rule: UTF-8 must be treated as a pipeline contract, not a terminal preference.

```
Recommended invariant:
HTTP/HTML -> Unicode -> UTF-8 files -> UTF-8 subprocesses -> UTF-8 gateway -> UTF-8 delivery

Anti-pattern:
Assume the Windows default code page is sufficient for AI-generated multilingual text.
```

10. Gateway and Telegram: From Local Tool to Remote Service

The Telegram integration materially changed the threat model. Before Telegram, the user was primarily interacting with a local process. After Telegram, a network-facing adapter could receive instructions, trigger agent work and deliver results. Hermes documentation therefore emphasizes authorization and allowlists for messaging platforms, and explicitly treats the bot token as a secret. [2], [8]

During the case, a gateway anomaly was narrowed by temporarily disabling the Telegram platform configuration. That step should be interpreted as fault-domain isolation rather than evidence of an intrinsic Telegram security flaw. Without a full original gateway trace, the correct conclusion is that platform configuration was an interacting component in the observed gateway failure.

For a personal deployment, Telegram should be treated as an external control plane: the bot token is a credential, the allowlist is an authorization boundary, and cron delivery is an outbound channel. These responsibilities should be monitored separately.

11. Security Threat Analysis

Table 3. Threats, surfaces, consequences and control principles.

Threat	Primary surface	Potential consequence	Control principle
Prompt injection / goal hijack	Web, files, email, tool results, Telegram content	Agent accepts untrusted instructions as task authority	Containerize execution; constrain tools; isolate untrusted content; require review for boundary-changing actions.

Hermes Agent on a Personal Computer

Threat	Primary surface	Potential consequence	Control principle
Excessive agency	Terminal, file tools, cron, messaging, scripts	Model can take more consequential actions than the task requires	Least privilege; narrow toolsets; read-only data; no broad host mounts; deterministic scripts for fixed tasks.
Credential exposure	Environment variables, config files, forwarded env	Token/API key becomes readable to code executing in a lower-trust context	Do not forward secrets by default; use scoped credential mechanisms; rotate leaked tokens.
Filesystem integrity	Bind mounts, writable workspaces, path translation	Incorrect path targets a different directory or a write crosses a boundary	Explicit path allowlists; read-only mounts; verify resolved path before writes; avoid ambiguous relative paths.
Supply-chain risk	Skills, plugins, MCP servers, Docker images	Third-party code executes with agent privileges	Review code; pin versions/images where possible; minimize installed extensions.
Gateway exposure	Telegram, dashboard, HTTP adapters	Unauthorized caller can dispatch work or receive output	Allowlist; loopback-only for local interfaces; explicit authentication for network-exposed surfaces.
Automation reliability	Cron jobs, catch-up scripts, delivery	Repeated autonomous errors amplify one configuration defect	Per-job health checks; idempotency; observable logs; manual trigger path; staged changes.
Encoding corruption	Windows locale, subprocess pipes, gateway readers	Data silently changes meaning or delivery fails	UTF-8 end-to-end; explicit encodings at file and subprocess boundaries; multilingual test cases.

The analysis aligns most directly with OWASP concepts of Prompt Injection, Sensitive Information Disclosure, Supply Chain risk and Excessive Agency. OWASP describes excessive agency as damaging action caused by excessive functionality, permissions or autonomy, including direct or indirect prompt injection. [16], [17] The 2026 OWASP Top 10 for Agentic Applications places these concerns within a dedicated agentic-security framework. [18]

12. Observed Architecture and Secure Target Architecture

The following logical diagram intentionally omits personal usernames, IP addresses, bot identifiers, file locations and credentials. It distinguishes the observed deployment pattern from the security target.

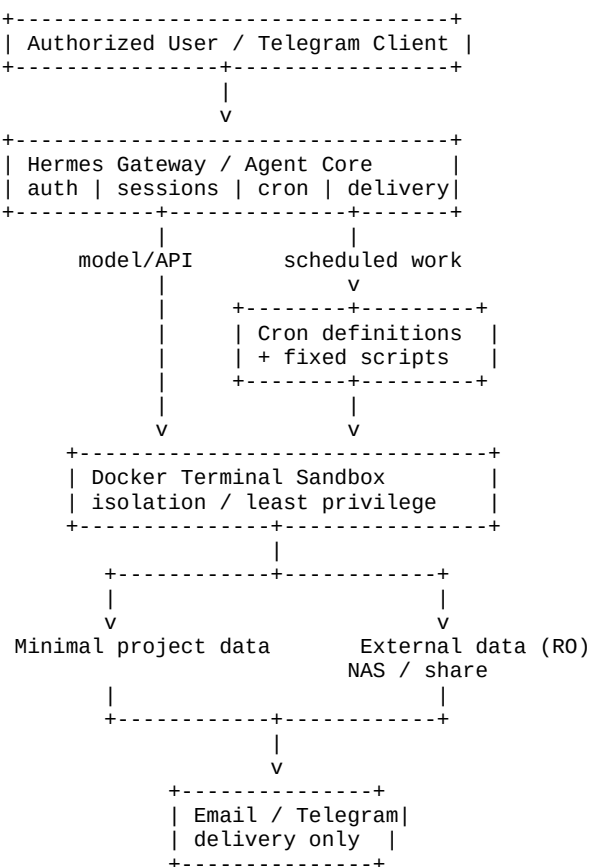


Figure 1. Logical architecture and explicit trust boundaries for the target state.

In the target state, each edge is an explicit trust boundary: messaging requires authorization; execution is containerized; data mounts are minimal and read-only where practical; secrets are scoped; cron scripts are deterministic; delivery channels do not become execution channels; and the agent does not receive a host-wide control path such as a writable Docker socket.

13. Engineering Lessons

- Agent security must be expressed as a system architecture, not merely as prompt instructions.
- OS isolation is fundamentally different from an approval dialog or denylist. The latter are useful controls for cooperative behavior but not hard containment. [2]
- Docker reduces blast radius only if mounts, environment forwarding, credentials and network exposure remain minimal. [3], [5]
- Windows path handling deserves first-class tests in agentic automation, especially when cron, PowerShell, Python, Node and Docker coexist.
- UTF-8 errors can cross from reliability into data-integrity and security domains because corrupted values can alter classification, routing or downstream actions.
- Cron jobs must be treated as production workloads: independent context, deterministic inputs, explicit outputs, idempotency and observable delivery.
- Telegram changes the trust model from local automation to a remote control surface and therefore requires explicit authorization and token hygiene. [2], [8]
- Incremental remediation - backup, one targeted change, syntax validation, functional test, delivery test - provides better fault localization than broad refactoring during incident response.

14. Recommended Future-State Control Set

Table 4. Recommended future-state controls.

Control area	Recommended control
OS-level boundary	Keep destructive or broad terminal/file operations out of the Windows host; prefer whole-process containment where untrusted inputs are involved.
Minimal filesystem exposure	Mount only required directories. Prefer read-only for reference data. Never mount the host root or unrelated personal directories.
Credential minimization	Do not use broad environment forwarding. Keep provider, Telegram, SMTP and other secrets out of prompts, source code and repositories.
Tool minimization	Disable unused tools and connectors. Separate advisory workloads from workloads that can modify systems.
Messaging authorization	Use explicit Telegram allowlists and protect bot tokens. Keep local dashboards loopback-only unless exposure is deliberate and authenticated.
Cron hardening	Give every job a stable working directory, explicit script, timeout, output location, delivery target and manual test procedure.
Path normalization tests	Test Windows host path, container path, absolute path, relative path and script invocation independently.
Encoding contract	Require UTF-8 at source files, subprocess capture, gateway pipes, generated reports and email/Telegram delivery.
Version reproducibility	Record Hermes release, Docker image tag, configuration checksum and test date for meaningful changes.
Observability	Monitor gateway liveness, last successful cron run, last error, delivery result and output freshness; distinguish execution failure from delivery failure.

15. Discussion: What This Case Contributes

This case contributes three practical observations to the emerging literature and practice of personal agent security. First, the transition from local execution to containerized execution does not eliminate risk; it moves the dominant failure surface from unrestricted host authority toward boundary configuration. Second, reliability defects - especially path translation and encoding corruption - can become security-relevant when they alter where an agent writes, what data it reads, what content it sends, or whether a scheduled control actually fires. Third, remote messaging and cron convert a single-user experimental agent into an asynchronous automation platform. That increases the importance of authorization, idempotency, observability and deterministic recovery.

The case should not be generalized statistically; it is a single practitioner deployment. Its value is architectural and methodological. It illustrates how agentic risk emerges from the composition of capabilities, interfaces and execution environments rather than from the language model alone.

16. Conclusion

The central lesson of this deployment is that an autonomous AI agent on a personal computer should be engineered as a security-sensitive automation platform. The relevant question is not simply whether the model is aligned or whether a command requires approval. The relevant question is which OS resources, files, credentials, network surfaces, scheduled tasks and external identities are reachable when the model makes a mistake or is influenced by untrusted content.

The resulting target architecture is therefore layered: authorized messaging enters a controlled gateway; the agent executes commands through a sandbox; filesystem exposure is explicit and minimized; scheduled jobs are independently testable; credentials are scoped; outputs are validated and delivered through separate channels; and the entire deployment is versioned and observable. This architecture is consistent with Hermes documentation, current OWASP agentic-security thinking, and Docker isolation practices, while also reflecting the concrete failure modes observed in this Windows case study.

References

- [1] NousResearch, "Hermes Agent Releases," v0.21.3 (v2026.9.14), GitHub, Sep. 14, 2026. [Online]. Available: <https://github.com/NousResearch/hermes-agent/releases/tag/v0.21.3>. Accessed: Sep. 24, 2026.
- [2] NousResearch, "Hermes Agent Security Policy (SECURITY.md)," GitHub. [Online]. Available: <https://github.com/NousResearch/hermes-agent/blob/main/SECURITY.md>. Accessed: Sep. 24, 2026.
- [3] NousResearch, "Security," Hermes Agent documentation. [Online]. Available: <https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/security.md>. Accessed: Sep. 24, 2026.
- [4] NousResearch, "Windows (Native) Guide," Hermes Agent documentation. [Online]. Available: <https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/windows-native.md>. Accessed: Sep. 24, 2026.
- [5] NousResearch, "Configuration," Hermes Agent documentation. [Online]. Available: <https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/configuration.md>. Accessed: Sep. 24, 2026.
- [6] NousResearch, "Scheduled Tasks (Cron)," Hermes Agent documentation. [Online]. Available: <https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/features/cron.md>. Accessed: Sep. 24, 2026.
- [7] NousResearch, "Windows UTF-8 console documentation," Hermes Agent documentation. [Online]. Available: <https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/windows-native.md>. Accessed: Sep. 24, 2026.
- [8] NousResearch, "Telegram," Hermes Agent documentation. [Online]. Available: <https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/messaging/telegram.md>. Accessed: Sep. 24, 2026.
- [9] NousResearch, "Docker Setup," Hermes Agent documentation. [Online]. Available: <https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/docker.md>. Accessed: Sep. 24, 2026.
- [10] Docker, "Bind mounts," Docker Engine documentation. [Online]. Available: <https://docs.docker.com/engine/storage/bind-mounts/>. Accessed: Sep. 24, 2026.
- [11] Docker, "Rootless mode," Docker Engine documentation. [Online]. Available: <https://docs.docker.com/engine/security/rootless/>. Accessed: Sep. 24, 2026.
- [12] NousResearch/hermes-agent, "[Bug]: Windows cron scheduler loses backslashes in script paths (C:\Users -> C:Users)," GitHub Issue #60857, Jul. 8, 2026. [Online]. Available: <https://github.com/NousResearch/hermes-agent/issues/60857>. Accessed: Sep. 24, 2026.
- [13] NousResearch/hermes-agent, "[Bug] Windows: subprocess env missing PYTHONUTF8=1 causes UnicodeEncodeError on non-ASCII output in CP936/legacy locales," GitHub Issue #31420, May 24, 2026. [Online]. Available: <https://github.com/NousResearch/hermes-agent/issues/31420>. Accessed: Sep. 24, 2026.
- [14] NousResearch/hermes-agent, "Bug: Desktop [gateway-crash] - GBK encoding kills gateway on Chinese Windows," GitHub Issue #83851, Aug. 11, 2026. [Online]. Available: <https://github.com/NousResearch/hermes-agent/issues/83851>. Accessed: Sep. 24, 2026.
- [15] NousResearch/hermes-agent, "`no\_agent` cron script stdout silently dropped on Windows (pythonw gateway) when output contains emoji / multi-byte UTF-8," GitHub Issue #42384, Jun. 8, 2026. [Online]. Available: <https://github.com/NousResearch/hermes-agent/issues/42384>. Accessed: Sep. 24, 2026.
- [16] OWASP GenAI Security Project, "LLM06:2025 Excessive Agency." [Online]. Available: <https://genai.owasp.org/llmrisk/llm062025-excessive-agency/>. Accessed: Sep. 24, 2026.
- [17] OWASP GenAI Security Project, "LLM02:2025 Sensitive Information Disclosure." [Online]. Available: <https://genai.owasp.org/llmrisk/llm022025-sensitive-information-disclosure/>. Accessed: Sep. 24, 2026.
- [18] OWASP GenAI Security Project, "Top 10 for Agentic Applications for 2026." [Online]. Available: <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>. Accessed: Sep. 24, 2026.
- [19] OWASP GenAI Security Project, "GenAI LLM Top 10." [Online]. Available: <https://genai.owasp.org/llm-top-10/>. Accessed: Sep. 24, 2026.

Appendix A - Public-Release Privacy Checklist

Before public release, verify that the research artifact does not disclose unnecessary personal, system, credential, or infrastructure information.

- No Telegram bot username, chat ID, user ID or token.
- No Windows username, absolute personal directories, private LAN addresses or SMB credentials.
- No SMTP password, API key, model-provider secret or session credential.
- No raw config.yaml, .env, SMTP configuration, session database or full gateway logs.
- No private ChatGPT conversation URLs in the public research artifact.
- All job identifiers are generalized to JOB-ASX, JOB-US, JOB-CATCHUP and JOB-SUPERMARKET.
- All personal paths are replaced with logical placeholders such as <WORKSPACE>.
- No identifiable machine name or other unnecessary system identifier.
- No screenshots containing private information, credentials, tokens, usernames or network details.
- Verify document metadata before publication and remove unintended personal, system, revision, or authoring information.
- Retain only intentional publication metadata, such as the author's name and academic/professional credentials.

Appendix B - Proposed Reproducibility and Validation Test Matrix

The following matrix defines reproducibility and validation tests derived from the practitioner case study. The "Actual Result" column records observed outcomes from the case study where applicable; it should not be interpreted as a universally reproducible result across all Hermes, Docker, Windows, or host configurations.

Table 5. Proposed reproducibility and validation tests.

Test ID	Test	Expected Result	Actual Result	Status
R-01	Container isolation	Host files inaccessible	Containerized agent access was constrained by the configured Docker execution boundary; host-side access was not treated as inherently safe merely because the agent was running in Docker.	PASS*
R-02	Read-only mount	Write denied	Read-only access was explicitly targeted for the NAS/shared workspace. Direct Windows drive binding did not provide the intended NAS isolation, and CIFS-based access required additional permission/session configuration before it could be considered a validated read-only control.	CONDITIONAL*
R-03	Cron execution	Job executes	Scheduled ASX, US-market and catch-up jobs were successfully executed and produced the expected operational outputs during validation.	PASS
R-04	Invalid path	Safe failure	Invalid and translated Windows/Docker paths were identified during testing; path translation failures did not establish permission to access the intended host location and required explicit path validation.	PASS*
R-05	Unicode output	UTF-8 preserved	Encoding and mojibake issues were observed in intermediate testing and subsequently corrected in the affected scripts/output path. Post-fix Python syntax validation and controlled output tests completed successfully.	PASS*

* Status reflects the observed practitioner environment and configuration, not a universal security guarantee. Reproduction on another system should independently validate the stated control.